

SOFIAA OS

Análisis Doctoral

T H E M I S

Governance & Ethics Engine

El tribunal constitucional de la inteligencia artificial operacional

Versión 1.0 — Sprints T-1 · T-2 · T-3
Julio 2026

1. Resumen Ejecutivo

THEMIS es el motor de gobernanza y ética de SOFIAA OS. Su nombre deriva de la diosa griega de la ley, la justicia y el orden divino — la que sostiene la balanza entre lo correcto y lo incorrecto. En el ecosistema de SOFIAA, THEMIS cumple exactamente esa función: evalúa cada respuesta del LLM y cada acción autónoma de HERMES contra un conjunto de políticas cognitivas declarativas, emite un veredicto estructurado e inmutable, y persiste ese juicio en Firestore para auditoría permanente.

A diferencia de los demás módulos del sistema — que optimizan la capacidad de SOFIAA para actuar más, recordar más, o generar más — THEMIS existe para poner límites. Es el tribunal constitucional que no puede ser silenciado, cuyo historial no puede editarse, y cuya evaluación corre con independencia del flujo principal de respuesta.

Principio constitucional: todo juicio de THEMIS queda registrado. La historia de la balanza no se edita, no se borra, no se silencia.

El motor fue implementado en tres sprints consecutivos (T-1, T-2, T-3) que construyeron, respectivamente: el núcleo de evaluación post-stream, la integración con HERMES como gate pre-ejecución, y el dashboard de administración de políticas en tiempo real.

2. Contexto y Motivación

Conforme SOFIAA OS adquirió capacidad de acción autónoma — a través de HERMES, que ejecuta tareas en sistemas externos como Monday.com y Slack — surgió una pregunta arquitectónica fundamental: ¿quién supervisa las decisiones de los supervisores?

Los sistemas de IA con capacidad de acción real enfrentan un problema conocido como alineación de valores: el modelo puede entender una instrucción perfectamente y aun así ejecutar algo que viola expectativas implícitas de seguridad, privacidad, o tono institucional. El Cognitive Policy Engine (Sprint D-D) resolvió la mitad del problema: inyectar restricciones en el system prompt para guiar al LLM. THEMIS resuelve la otra mitad: verificar que esas restricciones se cumplieron después de que el LLM respondió, y bloquear acciones antes de que se ejecuten.

La decisión de diseño más importante fue la separación de responsabilidades: THEMIS no genera texto, no razona con LLM, no predice. Evalúa de forma determinista en menos de 50ms usando detectores rule-based que no pueden ser engañados por el propio modelo que evalúan.

3. Arquitectura del Motor

3.1 Flujos de Evaluación

THEMIS opera en dos flujos distintos según el sujeto evaluado:

- Flujo T-1 (respuestas): `route.ts` → `evaluateResponse()` → `ThemisVerdict` → `Firestore`. Corre en `waitUntil()` post-stream, nunca bloquea la respuesta al usuario.
- Flujo T-2 (acciones): `hermes/execute` → `evaluateAction()` → `ThemisVerdict` → `Firestore`. Corre síncronamente antes de la ejecución; un veredicto 'block' aborta la acción.

3.2 Cognitive Policy Engine

El núcleo de políticas reside en `src/core/cognitive.policy.ts`. Cada política es un objeto `CognitivePolicy` con:

- `id` y `name`: identificación legible para humanos y para Firestore.
- `scope`: determina cuándo aplica la política (`global`, `extension`, `goal_active`, `unauthenticated`).
- `priority`: las políticas de mayor prioridad se inyectan primero en el `system prompt`.
- `constraints`: array de restricciones, cada una con su detector determinista y su instrucción natural que se inyecta al LLM.

3.3 Políticas Registradas

Política	Scope	Prior.	Constraints
Tono SOFIAA	Global	100	no_robotic · no_apologies · spanish_first
Privacidad y datos	Global	95	no_expose_keys · no_expose_internal_paths
Precisión Navegación	Global	90	exact_routes
JP Memorial	Extensión	85	empathy_first · no_price_pressure
Goal Activo	Goal activo	92	one_question_per_step · confirm_step_data
Usuario Anónimo	Sin sesión	88	gentle_auth_prompt
TEC BI Formal	Extensión	80	formal_language · data_grounded
Marketing Sofia	Extensión	80	actionable_ideas

3.4 Motor THEMIS — `src/core/themis.ts`

El motor opera en cuatro fases secuenciales para cada evaluación:

Fase 1: Resolución de overrides: Consulta Firestore (`system/themis`) para obtener qué políticas están desactivadas por admin. Cache de 60 segundos para evitar latencia.

Fase 2: Selección de políticas: `getPoliciesForContext()` filtra el registry según el scope y el contexto del request actual.

Fase 3: Detección de violaciones: `detectViolations()` corre todos los detectores de todas las políticas activas. Determinista, sin LLM, siempre < 10ms. Un detector roto nunca detiene la evaluación.

Fase 4: Veredicto y persistencia: Se construye `ThemisVerdict` con id único, timestamp ISO 8601, lista de violaciones, cadena de razonamiento legible, y severidad agregada. Se persiste con `set()` inmutable en Firestore.

4. Tipos y Contratos

4.1 ThemisVerdict

El veredicto es el ciudadano de primera clase de THEMIS. Una vez escrito en Firestore es inmutable — no se edita, no se sobrescribe. Contiene:

- `id` (UUID v4), timestamp (ISO 8601), date (YYYY-MM-DD para queries por fecha).
- `subject`: 'response' para evaluaciones post-LLM, 'action' para gates pre-HERMES.
- `approved`: false únicamente cuando `severity === 'block'`.
- `violations`: lista de `ThemisPolicyViolation` con `policyId`, `constraintId`, `evidence` (máx. 120 chars), e `instruction`.

- reasoning: cadena de razonamiento en español, legible para humanos, generada de forma determinista.
- policiesEvaluated: IDs de todas las políticas que participaron en la evaluación.
- severity: 'clean' | 'warn' | 'error' | 'block'. Solo 'block' impide ejecución.

4.2 Escala de Severidad

clean: Cero violaciones detectadas. Veredicto positivo.

warn: Violaciones de baja severidad. Respuesta aprobada, pero el veredicto queda registrado para auditoría.

error: Violaciones de alta severidad en respuestas. Aprobada (no bloquea el stream), pero disparará alertas en análisis posteriores.

block: Violaciones de alta severidad en acciones. La ejecución de HERMES se aborta. Solo aplica al flujo T-2.

4.3 ActionEvaluationRequest / Response

HERMES construye un ActionEvaluationRequest con actionId, actionType, actionPayload (el objeto completo de la acción), requestedBy (qué extensión la originó), userId, y context. THEMIS responde con ActionEvaluationResponse: el veredicto completo, approved (boolean), y conditions (para futuras restricciones condicionales).

5. Integración con HERMES — Sprint T-2

Sprint T-2 implementó el gate constitucional entre HERMES y el mundo externo. El flujo completo es:

- HERMES recibe una acción aprobada por el usuario en la cola (/hermes/cola/page.tsx).
- El executor (src/extensions/hermes/executor.ts) llama a POST /api/themis/evaluate-action con INTERNAL_SECRET antes de despachar.
- THEMIS evalúa el payload de la acción contra las políticas activas.
- Si approved === false (severity: 'block'), el executor registra la acción como 'vetada_por_themis' y no ejecuta.
- El historial de HERMES (/hermes/historial) muestra las acciones vetadas con el veredicto completo para transparencia.

El campo vetada_por_themis fue agregado al schema de HermesAction (src/types/hermes.ts) como campo opcional que contiene el ThemisVerdict completo. Esto permite que el historial sea autoexplicativo sin consultar una colección externa.

Ninguna acción de HERMES puede ejecutarse sin pasar por la balanza de THEMIS. Este invariante es el principio de seguridad más importante de SOFIAA OS.

6. Policy Override Dashboard — Sprint T-3

Sprint T-3 cerró el ciclo de gobernanza con la capacidad de modificar el comportamiento de THEMIS en tiempo real, sin deployar código. El sistema consta de tres componentes:

6.1 API de Overrides

GET /api/themis/policy-override devuelve todas las políticas del registry enriquecidas con su estado de override actual. POST /api/themis/policy-override acepta { policyId, enabled, note } y escribe en Firestore (system/themis → campo overrides). Tras una escritura exitosa, llama a invalidateOverridesCache() para que el próximo request lea el estado fresco.

6.2 Cache de 60 Segundos

El motor THEMIS mantiene un cache en memoria (_overridesCache) con TTL de 60 segundos. Esto balancea la latencia (evitar hit Firestore en cada evaluación) con la frescura del estado (un cambio de admin aplica en máximo 60 segundos). Si Firestore no responde, el cache anterior se reutiliza silenciosamente — THEMIS nunca falla por conectividad.

6.3 Dashboard /hermes/themis

Interfaz administrativa completa accesible desde el sidebar de HERMES. Muestra:

- Tarjeta por cada política del registry con nombre, ID, scope, y prioridad.
- Toggle on/off con campo de nota obligatorio al desactivar.
- Lista de todos los constraints con su severidad y la instrucción inyectada al LLM.
- Timestamp de última modificación y qué admin realizó el cambio.

La combinación de override en Firestore + cache + dashboard convierte a THEMIS en un sistema de gobernanza operativa real: un administrador puede responder a un incidente desactivando una política en segundos, sin necesidad de un redeploy.

7. Evaluación Post-Stream — Sprint T-1

La integración de THEMIS con el chat principal (src/app/api/chat/route.ts) sigue el patrón waitUntil() del Edge Runtime: una vez que el stream de respuesta termina y se ha enviado el último token al usuario, se lanza la evaluación asíncrona sin bloquear ni modificar lo que ya se entregó.

Esta decisión de diseño refleja una filosofía clara: THEMIS en T-1 es un sistema de auditoría, no de censura. No tiene capacidad de reescribir o retener una respuesta ya streaming. Su valor está en el registro: cada interacción queda evaluada, los patrones de violación se vuelven visibles en el tiempo, y futuras versiones pueden usar ese historial para ajustar los detectores o activar reintentos automáticos.

Persistencia: users/{userId}/themis_verdicts/{verdict.id} — colección subcollection por usuario.

Inmutabilidad: set() sin merge — un veredicto escrito no puede ser modificado.

Seguridad: logVerdict() nunca propaga errores — THEMIS no puede romper el stream bajo ninguna circunstancia.

8. Diseño Determinista — Sin LLM en la Balanza

Una decisión arquitectónica fundamental de THEMIS es que ninguna de sus evaluaciones usa un LLM. Todos los detectores son funciones puras de tipo (response: string, context: PolicyContext) => boolean. Esto garantiza cuatro propiedades críticas:

Velocidad: < 10ms para evaluación de respuestas, < 50ms para acciones (incluyendo la llamada HTTP a /api/themis/evaluate-action).

Determinismo: El mismo texto produce siempre el mismo veredicto. No hay no-determinismo estadístico del modelo.

No capturable: Un LLM no puede manipular a su propio evaluador hablando de cierta manera, porque el evaluador no lee lenguaje natural.

Independencia: THEMIS puede evaluar una respuesta del LLM sin que ese LLM esté disponible — los detectores funcionan aunque el proveedor de IA esté caído.

El tradeoff conocido es la cobertura semántica: detectores regex no capturan matices sutiles de tono o contexto. Sprint T-1 reconoce esto explícitamente en el código (confirm_step_data: detect: () => false — 'evaluación semántica difícil sin LLM — solo inyectar la instrucción'). La hoja de ruta contempla un modo híbrido donde detectores simples filtran el 90% de los casos y un clasificador ligero se invoca solo para los casos ambiguos.

9. Métricas y Rendimiento Operacional

THEMIS está diseñado para operar dentro de los siguientes parámetros:

Latencia de evaluación de respuesta: < 10ms (detectores rule-based en memoria)

Latencia de evaluación de acción: < 50ms (incluye HTTP round-trip a /api/themis/evaluate-action)

Latencia de escritura en Firestore: < 200ms (fire-and-forget, no bloquea el stream)

TTL de cache de overrides: 60 segundos

Límite de evidencia por violación: 120 caracteres

Políticas registradas en v1.0: 8 políticas, 16 constraints

Colecciones Firestore: users/{uid}/themis_verdicts (subcollection por usuario)

Colección de admin: system/themis (overrides globales)

10. Seguridad y Control de Acceso

THEMIS opera sobre dos perímetros de seguridad distintos:

10.1 Motor-to-Motor (INTERNAL_SECRET)

El endpoint POST /api/themis/evaluate-action está protegido por INTERNAL_SECRET — una variable de entorno compartida solo entre los motores internos de SOFIAA. Ningún cliente externo puede llamar directamente a la balanza. HERMES autentica con Bearer INTERNAL_SECRET. Si la variable no está configurada, el endpoint rechaza todas las solicitudes con 401.

10.2 Dashboard Admin (Firebase Auth)

GET y POST /api/themis/policy-override requieren un token válido de Firebase Auth con claims de admin (verificados por Firebase Admin SDK). El dashboard /hermes/themis valida la sesión del usuario antes de cargar y está integrado al sidebar de HERMES, que ya tiene su propio RBAC.

10.3 Veredictos como Evidencia

Los veredictos en Firestore constituyen evidencia auditable de cómo SOFIAA se ha comportado. Las Firestore Security Rules restringen `users/{uid}/themis_verdicts` a lecturas del propio usuario y escrituras solo desde Firebase Admin SDK (servidor). Esto garantiza que ni el usuario final ni un atacante puedan alterar el historial de la balanza.

11. Estado Actual y Hoja de Ruta

11.1 Implementado (v1.0)

- T-1: Motor de evaluación post-stream con 8 políticas y 16 constraints.
- T-1: Persistencia inmutable en Firestore con ThemisVerdict completo.
- T-1: Integración `waitUntil()` en `chat/route.ts` — nunca bloquea el stream.
- T-2: Gate pre-ejecución en HERMES executor — ninguna acción sin veredicto.
- T-2: endpoint `/api/themis/evaluate-action` con `INTERNAL_SECRET`.
- T-2: Campo `vetada_por_themis` en `HermesAction` y visualización en historial.
- T-3: Cache de overrides con TTL 60s + invalidación explícita.
- T-3: API REST GET/POST `/api/themis/policy-override`.
- T-3: Dashboard `/hermes/themis` con toggle y notas de auditoría.

11.2 Hoja de Ruta Futura

- T-4: Analytics de veredictos — panel de métricas históricas (tasa de violaciones por política, tendencias).
- T-5: Modo híbrido — clasificador ligero para constraints semánticos complejos.
- T-6: Alertas activas — notificar al admin cuando la tasa de violaciones supera umbral configurable.
- T-7: Exportación de historial — APOLO integration para incluir veredictos en reportes ejecutivos.
- T-8: Políticas dinámicas — agregar nuevas políticas desde el dashboard sin modificar el código fuente.

12. Conclusión Doctoral

THEMIS representa el componente de mayor importancia filosófica en la arquitectura de SOFIAA OS. Mientras que todos los demás módulos amplían la capacidad del sistema — más memoria, más acción, más predicción — THEMIS es el único diseñado para contener.

La decisión de implementar un evaluador determinista sin LLM, persistencia inmutable de veredictos, y un gate constitucional pre-ejecución refleja una comprensión madura del riesgo real en sistemas de IA con acción autónoma: el peligro no está en lo que el modelo no puede hacer, sino en lo que puede hacer sin supervisión. THEMIS convierte la supervisión en un artefacto de ingeniería, no en una promesa de diseño.

La integración completa de los tres sprints — evaluación de respuestas (T-1), gate de acciones (T-2), y administración en tiempo real (T-3) — construye el fundamento sobre el cual SOFIAA puede escalar su autonomía con confianza. Cada nueva capacidad que se añada al sistema pasa, inevitablemente, por la balanza.

SOFIAA puede actuar porque THEMIS observa. Puede escalar porque THEMIS audita. Puede ser confiable porque THEMIS no puede ser silenciada.

— Equipo SOFIAA OS · Julio 2026